

```
1 import makemore
2 import torch
3 import matplotlib.pyplot as plt
4
5 #read in the file of names
6 #python是从当前项目的根目录开始找文件的，所以这里的路径是相对于当前目录的
7 words = open('Neural Network_Zero to hero/makemore/names.txt', "r").read().splitlines()
8
9 #这里是打印数组前十个元素
10 words[0:10]
11
12 #声明一个空字典b，用来存储二元组的计数
13 b = {}
14
15
16 #第一行的含义是：遍历数组words的前一个元素（也就是第一个元素），并将其赋值给变量w
17 for w in words:
18
19     chs = ['<S>'] + list(w) + ['<E>']
20     #这里是将字符串w转换为一个字符列表，并在前后加上<S>和<E>，表示开始和结束
21
22     #解释一下，这里涉及的语法：
23     # zip()函数是将两个可迭代对象（这里是w和w[1:]）打包成一个个元组，然后返回由这些元组组成的列表。
24     # w[1:]表示从w的第二个元素开始到最后一个元素
25     # 这里的w是一个字符串（也就是一个名字），zip实际上把他们当成了一个个字符来处理
26     # 所以总的来说，zip(w, w[1:])的作用是将字符串w中的每个字符和它的下一个字符配对，形成一个个二元组
27     # 然后ch1,ch2就是两个参数，每一次for循环都会从zip返回的二元组中取出一个二元组，并将其解包赋值给ch1和ch2
28     # 然后zip指针指向下一个元组，再继续讲下一个元组赋值给ch1和ch2，直到zip返回的所有元组都被遍历完
29     # 这里w肯定比w[1:]多一个字符，zip是会在较短的可迭代对象耗尽时停止迭代的。
30
31
32
33     for ch1,ch2 in zip(chs, chs[1:]):
34
35         #解释代码：
36         # 这里的bigram是一个元组，包含了两个字符ch1和ch2
37         # 元组都是有序的。可以下标访问的序列list
```

```
38     # 但是元组是不可变的，也就是说一旦创建了一个元组，就不能修改它的内容
39     bigram = (ch1, ch2)
40
41     #解释代码：
42     b[bigram] = b.get(bigram, 0) + 1
43
44     #print(ch1, ch2)
45
46 #这里不打印了
47 #在下面引入Tensor的处理之后再打印
48
49 #换行
50 print('=====')
51
52
53
54 #=====这里引入torch的Tensor，使用Tensor处理名字信息=====
55 #并且做出一个核心改进：把开始符和结束符，合并成一个符号
56
57
58 #先引入一个字典。我们希望赋予每一个可能出现的符号一个唯一的整数
59 chars = sorted(list(set(''.join(words))))
60 #介绍这里用到的语法：
61 #   ''.join(words)是将words数组中的所有字符串连接成一个长字符串。是''空串调用的，所以空串是列表内字符串的连接符
62 #   set()函数是将一个可迭代对象转换为一个集合，集合是无序的，并且不允许重复元素
63 #   list()函数是将一个可迭代对象转换为一个列表
64 #   sorted()函数是将一个可迭代对象排序后返回一个新的列表，默认是升序排序。这里被排序的对象是字符。所以就是a-z的顺序
65
66
67
68 #这里是为了给刚刚创建的chars列表中的每一个字符分配一个唯一的整数索引。
69 stoi = {ch: i+1 for i, ch in enumerate(chars)}
70 #这里涉及的语法：
71 #   enumerate(chars)会返回一个枚举对象，它会生成一系列的元组，每个元组包含一个索引和对应的字符。
72 #   元组的格式是：先是索引，然后是字符
73 #   所以i是索引，ch是字符。
74 #   然后调用for循环的是ch: i这个表达式。所以就是循环给每一个字符分配一个唯一的整数索引。最终生成一个字典stoi
```

```
75
76
77
78 #这里是往stoi字典里添加两个新的键值对，分别是'<S>'和'<E>'，它们的值分别是26和27
79 #传入中括号内的字符串是键，等号右边的数字是值。
80 #如果stoi字典中已经存在这些键，那么它们的值会被更新为新的值；如果不存在，那么就会添加新的键值对。
81 stoi['.'] = 0
82
83
84 #再声明一个字典itos，它是stoi的逆映射，也就是将整数索引映射回字符。
85 itos = {i: ch for ch, i in stoi.items()}
86
87
88 #这里再创建一个28*28的二维张量b，初始化为0。这个张量用来存储二元组的计数。
89 #这里的28是因为我们有26个字母，加上<S>和<E>，一共就是28个符号。
90 #那么为什么是二维？
91 # 因为我们要统计的是二元组，也就是两个字符的组合。
92 # 每一个字符都可以是28个符号中的一个，所以我们需要一个28*28的矩阵来表示所有可能的二元组。
93 # 二维数组的第一行，表示以'.'字符开头的二元组的出现频率。第二行表示以'a'为开头的二元组的出现频率
94 N = torch.zeros((27, 27), dtype=torch.int32)
95
96
97
98 #进入循环开始统计
99 for w in words:
100     chs = ['.'] + list(w) + ['.']
101     for ch1, ch2 in zip(chs, chs[1:]):
102         ix1 = stoi[ch1]#这里stoi是字典，字符是id，返回的是key。ix1就是字符对应的数字标识
103         ix2 = stoi[ch2]
104         N[ix1, ix2] += 1
105
106
107
108 #尝试展示一个图形化窗口显示二维数组的内容
109 plt.figure(figsize=(27,27))
110 plt.imshow(N, cmap='Blues')
111 for i in range(27):
```

```
112     for j in range(27):
113         chstr = itos[i] + itos[j]
114         plt.text(j, i, chstr, ha='center', va='bottom', color='gray', fontsize=6) # 字符放格子下半部
115         plt.text(j, i, N[i,j].item(), ha='center', va='top', color='black', fontsize=6) # 数字放格子上半部
116 plt.axis('off')
117 plt.show()
118
119
120
121 #=====下面开始正式生成随机名字=====
122
123
124 #第一行进行归一化:
125 first_row = N[0].float() # .float() 是方法, 记得加括号: 把 N[0] 转换成浮点类型
126 first_row = first_row/first_row.sum()
127 #创建一个随机生成器对象Random
128 #涉及的语法:
129 # 这里得到的是一个Generator对象, 输入种子是214783647
130 Random = torch.Generator().manual_seed(214783647)
131 #利用这个生成器, 获取一个随机数字
132 #语法知识:
133 # 传入的第一个参数是一个Tensor。里面储存的是一串数字。数字的数量相当于选项的数量。数字的大小会被拿来计算相对概率
134 # num_sample表示, 要随机生成多少个数字
135 # replacement = True表示, 可以重复选择之前选过的选项
136 # generator表示, 用来生成随机数的随机数生成器
137 index_x = torch.multinomial(first_row,num_samples=1,replacement=True,generator=Random)
138
139
140
141 #所以上面这一部分的代码就实现了一个功能:
142 # 根据第一行的各种双元字母组合, 比如 (a, b), 的出现频率, 使用随机数生成器生成一个数字
143 # 这个数字表示选择第一行的哪一种情况。
144
145
146 #刚刚只是选择了开头的符号。现在我们完全进行一次名字的生成
147
148 #定义一个用于名字生成的函数
```

```

149 def name_generate (N):
150     #希望它传进来的参数是一个二维的Tensor数组
151     #我们这里希望进行一次简单的优化。一次性计算完每一行的概率分配
152     P_arrays = N.float()
153     P_arrays = P_arrays/P_arrays.sum(1,keepdim = True)
154
155     idx = 0
156     name = ''
157     while True:
158         P = P_arrays[idx]
159         idx = torch.multinomial(P,num_samples=1,replacement=True,generator=Random).item()
160         name += itos[idx]
161         if idx == 0:
162             break
163     print(name)
164
165
166 for i in range(10):
167     name_generate(N)
168
169
170 #=====定义loss=====
171 #上面成功实现了：根据输入的字符集训练：已知当前字符，然后按照概率去生成下一个字符
172 #接下来，我们希望找到一个方式，去评估该模型的好坏。从而进行‘训练’
173 #什么叫做评估该模型的好坏？
174 # 1.我们当前的模型，是根据names文件里面的名字，统计了各个二元组出现的概率
175 # 2.我们模型学到的是：给定当前字符，下一个字符出现的概率分布
176 # 比如当前字符是'a'，模型可能输出：下一个是'b'的概率0.3，etc。所有可能字符各有一个概率。
177 # 3.评估就是：拿真实的名字来对比，看经过names字符集训练的模型给真实出现的那个下一个字符分配了多高的概率。
178 # 4.比如Andrej是一个真的名字，作为例子：
179 # 这个名字可以分成下面这堆二元组：A->n, n->d, d->r, r->e, e->j
180 # 模型就会输出这些二元组的概率。
181 # 这些二元组的概率越大，说明模型效果越好。
182 # 因为输入是真实的名字，概率越大，就说明模型越有可能输出Andrej这个真实名字，所以效果越好
183
184
185 #Andre i在这里定义的Loss函数是这样的：

```

```
186 # 1.我们从names训练集，拿出所有名字，拆分成约22万个二元组——当然会有重复
187 # 2.然后从P_arrays里面取出，所有二元组对应的概率，然后相乘得到乘积L
188 # 3.因为P里面的概率都小于一，乘积的结果相当接近0。取ln之后L会变成比较大的负数
189 # 4.所以再取一次负号变成正数。
190 # 5.那么这样得到的Loss函数，也是越小表示模型效果越好。
191
192
193
194 #奇怪的地方：既然我们的Bigram本身就是从names统计出P_arrays的概率的，那么再拿names训练集计算Loss是不是有点循环了？
195 # 1.用训练集来算损失，得到的是“训练损失”。它衡量的不是“模型在未知数据上好不好”，而是“模型对已经见过的数据拟合得怎么样”。
196 # 2.训练神经网络时，每一步都要算“当前参数下模型在训练集上的损失”。
197 # 后用这个损失求梯度、更新参数。所以“在训练集上算损失”是训练过程的必要环节，不是最终评估。
198 # 3. 真正评估模型好坏，需要另留一份“测试集”或“验证集”。那部分数据不参与训练，只用来看模型能不能泛化到没见过的东西。
199
200 print('开始进行Bigram的Loss计算')
201
202
203 # (1) 复用上面已经算好的 N，得到概率表
204 # （就是 name_generate 里面那个 P_arrays，只是那边是局部变量，出了函数就没了，所以这里要再算一次）
205 P_arrays = N.float() / N.sum(1, keepdim=True).float()
206
207
208 # (2) 算 loss: 不用再重新遍历 22 万个二元组！
209 # 因为 N[i,j] 本身就是二元组 (i,j) 出现的次数，
210 # 所以"把同一个二元组的 log 概率重复加 N[i,j] 次"，等价于"乘一次 N[i,j]"
211 # 这样只需要处理 627 个非零格子，结果和逐条累加 228146 次完全一样
212 # mask 是为了跳过 N = 0 的格子，否则 log(0) = -inf 会让结果变成 nan
213 mask = N > 0
214 n = N.sum()
215
216 loss_bigram = -(N[mask].float() * P_arrays[mask].log()).sum() / n
217 print(f'Bigram 的 Loss {loss_bigram.item():.4f}')
218
219
220
221 #=====尝试进行类似神经网络那样子的训练=====
222 #create the training set of bigrams
```

```
223 # (1) 上面不是已经根据words里面的名字, 得到了二元组的概率分布矩阵了吗? 那个不就是训练吗? 这里的训练是要搞什么?
224 #     1.上面也是一种学习,但它靠的是直接统计,不涉及神经网络和梯度下降,而且无法推广到复杂模型。
225 #     2.我们下面要做的,是像micrograd一样,初始化一个带有一堆随机取值的参数w, b的MLP
226 #     3.然后把words的名字们当做是训练集,去训练这个神经网络
227 #     4.最后得到的效果应该是和上面的Bigram差不多。但是训练方法更高级了
228
229
230
231 # (2) Andrej希望在这里训练的神经网络是这样的: -----这里理解错了,看下面一点为准
232 #     1.功能上和Bigram是一样的,统计(或者说学习)各种二元组出现频率,然后根据上一个字符输出下一个字符
233 #     2.输入层是27个神经元,代表当前字符是什么。输出层也是27个神经元,表示下一个字符应该是什么
234 #     3.我们实际要做的训练,就是调整神经网络的参数w和b,使得生成的二元组尽可能和训练集的二元组,概率分布相似
235
236
237
238 # (2.5) Andrej要做的神经网络是这样的: 单层线性层 + softmax"的结构
239 #     1.只有两层。一个是输入层,一个是线性层。参数w和b在输入层和线性层之间
240 #     2.线性层,在这种结构下就同时作为输出层。
241 #     3.同时,这种结构只有一套参数,也就是只有一个学习层
242 #     4.我们希望的效果是这样的:
243 #         根据输入的向量-----也就是二元组的首个字母,输出它的下一个字母的可能性概率分布。
244 #         本质和上面Bigram查表的形式差不多
245 #     5.所以这里学习的本质就是调整w矩阵的参数,使得算出的概率分布尽可能和上面的Bigram的结果差不多
246
247
248
249 # (3) 这里实际上做的是:
250 #     把所有名字拆成二元组。
251 #     然后根据二元组的字符内容和先前定义的字典stoi,把字符二元组变成数字二元组。比如(a,b)--->(1,2)
252 #     然后二元组的第一个,存入xs数组,第二个存入ys数组。并且把xs和ys数组变成Tensor
253 #     并且,注意到这样处理的核心性质:同一个下标对应的xs内元素和ys内元素,构成原来的一个二元组
254 #     而且二元组们显然是允许重复的,毕竟它们出现的频率正是我们要统计的内容。
255 #     比如遍历words内的名字的时候,可能出现很多次(a,b)。
256
257 xs,ys = [],[]
258
259 for w in words:
```

```
260     chs = ['.'] + list(w) + ['.']
261     for ch1, ch2 in zip(chs, chs[1:]):
262         ix1 = stoi[ch1]
263         ix2 = stoi[ch2]
264         xs.append(ix1)
265         ys.append(ix2)
266
267     xs = torch.tensor(xs)
268     ys = torch.tensor(ys)
269
270
271 # (4) 上面说了我们希望做一个什么样的神经网络。这里我们需要处理一下输入
272 #     1.前面学的micrograd的MLP，只能接受一串数字。也就是Tensor张量。我们需要把二元组字符变成一串向量
273 #     2.所以我们这里把字符通过字典变成数字，然后采用one-hot编码。
274 #     3.比如b的key是2，那么传入神经网络的就是：一个27位的数组，下标为2的元素值为1，别的地方都是0
275 #     4.假设我们上面的循环最终得到的向量xs和ys长度最终是：n。
276 #     5.下面方法实际就是：把每一个xs和ys的分量，扩展成一个长度为27的一维数组
277 xs_one_hot = torch.nn.functional.one_hot(xs, num_classes=27).float()
278 ys_one_hot = torch.nn.functional.one_hot(ys, num_classes=27).float()
279
280
281
282 #创建一个名为 W 的 PyTorch 张量，形状为 27 行 27 列，元素是随机数。
283 #randn函数生成的随机数服从标准正态分布
284 # (5) 为什么这里的生成的W参数矩阵规格要求是27*27?
285 #     1.前面经过热位编码之后，xs的数据变成了这样：总长度为n，每一个元素都是长度为27的Tensor
286 #     2.这行生成的随机数矩阵也是：27*27个随机数
287 #     3.因为输入层有27个神经元，理论上来说，这一层神经元，每一个有多少个输出端，第二层就应该有多少个神经元
288 #     4.比如第二层有5个神经元，那么第一层的每一个神经元就应该有五个输出。每一个输出都有一个参数w
289 #     5.那么这种情况就应该是27*5个参数作为随机数
290 W = torch.randn((27,27), requires_grad=True)
291
292 #将对应的参数和输入相乘：
293 first_layer_out = xs_one_hot @ W
294
295
296 # (6) 二维Tensor讲行矩阵乘法的时候，怎么确定哪个是行哪个是列？
```

```

297 # 1.比如拿上面做例子：假如第二层有5个神经元——因为Andrej取的27*27两个维度一样，不好区分
298 # 2.那么拿到的xs热位矩阵规格是这样的：
299 #     第一层有n个元素，每一个元素是长度为27的张量。
300 #     每一行代表的就是：一个元组对应的输入向量
301 # 3.拿到的W参数矩阵是这样的：
302 #     第一层有27个元素，每一个元素是一个长度为5的张量。
303 #     一列就代表输入层的27个神经元，对于第二层的一个神经元在连接上的参数们w
304 # 4.我们期望的结果是这样的：
305 #     得到一个n*5的张量。第一层有n个元素，每一个元素是长度为5的张量
306 #     含义是：n个输入值代表我们前面读入了n个元组。
307 #         长度为5的张量表示，每一个输入的张量，会给第二层每一个神经元带来一个输出，所以一共就是5个
308 # 5.所以，我们采用行向量的思路来看这次矩阵乘法：
309 #     行向量：假设C = AB
310 #     那么：C 的每一行，等于 A 的对应行，与 B 的每一列做点积。
311 # 6.放到这里来看：
312 #     结果矩阵的第k行，就是xs热位矩阵的第k行和参数矩阵每一列逐个点乘的结果
313 #     那么，结果矩阵的第k列就是：某一个输入向量，作为神经网络的输入的时候，第二层五个神经元接收到的输入
314
315
316
317 # (7) 注意到，神经网络的输出值是有可能负的：——这层处理叫做softmax
318 # 1.我们是这样得到神经网络的输出的：
319 #     当前神经网络结构是单层的，也就是说，输入的向量，经过一次和参数们的计算，就进入第二层了
320 #     第二层也就是输出层。
321 # 2.输入的列向量都是一位热位编码的，经过w的运算，有可能最终输出给第二层的，也就是神经网络输出，会是负数
322 # 3.我们希望对这些负数做一些处理。把他们映射成正数，然后把这些浮点正数，当做频率处理，计算最终概率分布
323
324
325
326
327 # (8) 这里的n个二元组，有很多是重复的二元组。
328 # 1.这里的n大约是22万。但是二元组种类只有27*27种。所以其实神经网络进行了很多次重复运算
329 # 2.所以得到的probs矩阵，也有很多重复项
330 counts = first_layer_out.exp()#这里得到的是一个n*27的矩阵
331 probs = counts/counts.sum(1,keepdim=True)#这里算出来的结果是一个二维矩阵
332 num = xs.nelement()#这句话的意思是：计算xs这个张量内部元素数量，然后赋给num
333 loss = -probs[torch.arange(num).vs1.log().mean()]

```

```
334 #涉及的语法:
335 # torch.arange(num),表示生成的一个大小为num的张量,并且内容是从0到num-1.
336 # prob 【】 二维张量里面传入两个一维张量,是在高级索引:
337 #
338
339
340
341 # (9) 进行梯度下降:
342 learning_rate = 50.0
343 print('开始进行梯度下降')
344
345 #学习两百次
346 for k in range(2000):
347
348     #这一步进行前向传播
349     first_layer_out = xs_one_hot @ W
350     counts = first_layer_out.exp()
351     probs = counts / counts.sum(1, keepdim=True)
352     loss = -probs[torch.arange(num), ys].log().mean()
353
354     #重置grad防止偏差
355     W.grad = None
356     loss.backward()
357     W.data += -learning_rate * W.grad
358
359     #选择性打印Loss函数
360     if k % 200 == 0 or k == 1999:
361         print(k, loss.item())
362
```